

level '1,' indicating that the system is busy.

When the FSM is in the Idle state and the latch data pin LD is at logic level '1,' the data byte to be transmitted is appended with a logic bit '1' and assigned to the transmit register Tx_Reg. The busy flag is set to logic level '1' and the FSM goes to the Load_Tx state. In case the latch data pin LD is at logic level '0,' the TxBusy flag is set to logic level '0,' indicating that there is no data to be transmitted.

When the FSM is in the Load_Tx state and the transmit clock TopTx is at logic level '1,' the FSM moves to the next state Shift_Tx. The start bit [Tx_Reg[0]] is set to logic level '0' and the transmit bit count TxBitCnt is assigned a value of 9 (count of start, data and the parity bits to be transmitted). The transmitter FSM is designed using odd parity (Boolean constant parity = True and Boolean constant even = False). The same can be changed as per the design requirement. Since, the parity chosen is odd, value of Tx_Par is '1.'

When the FSM is in the Shift_Tx state and the transmit clock TopTx is at logic level '1,' the value of TxBitCnt is decremented by '1.' The Tx_Par bit is built over eight clock cycles and is generated by XORing the Tx_Par bit with the second LSB bit of the transmit register Tx_Reg. The Tx_Reg is then right shifted and '1' is appended as the MSB. The process is repeated till TxBitCnt is '1.' When the value of TxBitCnt equals '1,' we get the final value of Tx_Par bit which is then transmitted later. The FSM goes to the Stop_Tx state if the system is designed without using parity, otherwise it goes to the Parity_Tx state and the parity bit is assigned to the LSB of the transmit register. Since in our case we are using odd parity, the FSM goes to the Parity_Tx state.

In the Parity_Tx state, when the transmit clock TopTx is at logic level '1,' the stop bit is assigned to the LSB of the transmit register Tx_Reg and the FSM goes to the Stop_Tx state. In the Stop_Tx state, the stop bit is transmitted and the FSM goes to the Idle state.

The process that implements the

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download on source.efymag.com

receiver FSM (Rx_FSM) also has reset (RST) and clock (CLK) inputs in the sensitivity list. If RST is at logic level '1,' all the bits of the receive register (Rx_Reg) and data byte received (Dout) are made zero. The receive bit count (RxBitCnt), receive error (RxErr), receive ready (RxRdyi) and the receive parity (Rx_Par) are all cleared to zero and the FSM is made to go to the Idle state.

If the RST pin is at logic level '0,' on the rising edge of the clock CLK, it is checked whether the receive ready signal (RxRdyi) is at logic level '1' or '0.' If it is at logic level '1,' it indicates that the word has been received and the receive error flag (RxErr) and the receive ready signal (RxRdyi) are made zero.

After that the receiver FSM is implemented. If the FSM is in the Idle state, the receive count bit (RxBitCnt) is set to zero. Then the parity bit is checked. If the parity is even parity, the receive parity bit (Rx_Par) is set to zero, else it is set to one. If the receive signal (Rx_r) is zero, the FSM goes to the next state, that is, the Start_Rx state. When in the Start_Rx state, the FSM waits for the first data bit. When the receive clock (TopRx) is at logic level '1' and the Rx_r signal is still at logic level '1' then a warning is issued and the FSM goes to the RxOVf state, else the FSM goes to the Edge_Rx state.

RxOVf state is the error-handling state that reports the error and sends the FSM to the Idle state when Rx is at logic level '1.' In the Edge_Rx state, if the receive clock (TopRx) is at logic level '1,' the FSM goes to the Shift_Rx state if the RxBitCnt is not equal to eight. If the value of receive bit count (RxBitCnt) equals eight, the FSM goes to Parity_Rx state, if the parity is used in the receiver design, else it goes to the Stop_Rx state. In the Shift_Rx state, when the receive clock (TopRx) is at logic level '1,' the value of receive bit

count (RxBitCnt) is incremented by one and the receive register (Rx_Reg) is shifted right by including the received bit as the MSB.

The Rx_Par bit is updated by performing the XOR operation on the received bit and the initial parity bit. Next, the FSM goes to the Edge_Rx state. The FSM moves between the Edge_Rx and the Shift_Rx states till the receive bit count is equal to eight.

In the Parity_Rx state, the parity bit is sampled. When the receive clock (TopRx) is at logic level '1,' it is checked whether Rx_Par and Rx_r bits are the same. In case they are the same, then the FSM goes to the Parity_2 state, else it goes to the error state RxOVf.

In the Parity_2 state, when the receive clock (TopRx) is at logic level '1,' the FSM goes to the Stop_Rx state. The Parity_2 state basically generates a wait period of half the bit time. In the Stop_Rx state, when the receive clock (TopRx) is at logic level '1,' if the Rx_r signal is high then the contents of the receive register (Rx_reg) are sent to the receiver output pins (Dout), the receiver ready signal (RxRdyi) is set to logic level '1' and the FSM goes to the Idle state. The FSM then reports the character received in decimal after suitable conversion. If the Rx_r signal is not at logic level '1,' the FSM goes to the error state RxOVf. The last state implemented is the case that when the FSM is in any other state, then it should go to the Idle state.

Conclusion

This part described how to establish an RS232 communication link using an FPGA. In the next part, we will discuss how to manage clocks using clock managers in FPGA to generate a clock of the desired frequency from a given input clock.

To be continued...

Varsha Agrawal is a scientist at Laser Science and Technology Center (LASTEC), a premier DRDO lab working in the field of laser-based defence systems. She has more than 13 years of R&D experience in the design and development of a variety of systems for defence-related applications. She has authored two books and published more than 20 research papers and technical articles